Andres Trujillo
CS 410 Software Reverse Engineering
May 17, 2026

CS 410 Assembly to C++ Activity Deliverable

Assembly Code	C++ Code	Explanation of Functionality

`movl -8(%rbp), %eax`
`sall $3, %eax`
`subl $3, %eax`
`movl %eax, -4(%rbp)`
	`int y = (y << 3) - 3;`	This code uses an offset of -8 to get an integer(Operation suffix of 'l' for long from the assembly instruction 'movl' means it's a 32 bit integer or a 32 bit floating point) off the stack and shifts left by a constant value of '3' and subtracts a constant value '3'. Note: If the 'l' was a 'q' for quad it would be a 64-bit integer. Also, because the base pointer is 'rbp' which is 64-bit addressing, we can deduce 'l' is an integer due to the arithmetic operations performed that use integers to manipulate the final value because a 'float' data type primitive or a double has 8 bytes which translates to 64 bits. And 'eax' is a 32-bit register so the compiler created the destination instruction as 'eax' which means we save the value found at the -8 offset and place it within 'eax' register contents.

...continued on the next page.

Assembly Code	C++ Code	Explanation of Functionality

movl -8(%rbp), %eax sall $2, %eax subl $1, %eax leal 7(%rax), %edx testl %eax, %eax cmovs %edx, %eax sarl $3, %eax movl %eax, -4(%rbp)	int x = (x << 2) - 1; int y = x >> 3;	We grab an int from offset -8 and move it to the %eax register, then left-shift it by 2 to multiply by 4. We subtract 1, then the compiler uses leal to prep a bias of 7 in %edx. It runs testl to do a bitwise AND on the value—this sets the Sign Flag if it's negative. If that flag is tripped, cmovs kicks in to add the bias so that the final right-shift (sarl $3) rounds toward zero like C++ expects for division. The result finally gets saved to offset -4.
movl -8(%rbp), %eax leal 7(%rax), %edx testl %eax, %eax cmovs %edx, %eax sarl $3, %eax movl -8(%rbp), %edx sall $2, %edx addl %edx, %eax movl %eax, -4(%rbp)	int y = (x >> 3) + (x << 2);	This performs two operations on the input x: it divides it by 8 (using the same rounding logic as above) and then adds the result to x multiplied by 4 (left shift by 2). The final sum is stored at -4.